

Managing Scanned Books

Rohit Goswami · 2023-06-26 · workflow · ramblings

Where I consider digitizing colored content.

Background

It so happened that I recently acquired a rather unique book which (somewhat oddly) had no corresponding digital variant [^fn:1]. I've discussed how I modify books and papers [for consumption on my Kobo Aura HD in the past](#), but this comes before that. Much of this is essentially a plug for [Scantailor Advanced](#) [^fn:2].

Goals

Starting with the initial output `pdf` files from a scanner's "scan to me" function [^fn:3] the goal is to have all the bells and whistles of a well-done digital book:

Fixing orientations

This is meant to deal with rotations mostly, skew is later

Splitting pages

Since most scanners with books take two pages at a time

Handling skew

Deskewing works to reorient pages for visibility

Reworking content selections

The automated selection worked well for this, and is especially useful if there are artifacts

Adding back margins

For padding post selection

Output determination

600 DPI is generally enough, and "mixed" mode works great for images, the one thing to add is that "Higher search sensitivity" for picture shapes is pretty useful

These six steps can be carried out with Scantailor Advanced, which has a very intuitive GUI for the same but there are but two more things:

- Optical Character Recognition (OCR)
- A proper Table of Contents

We will also have to first get high resolution images out of the `pdf` files. For starters our project might look like this:

```
> ls
appendix.pdf      chap6.pdf          images
chap2_chap3.pdf  chap7.pdf          toc_chap1_intro.pdf
chap4.pdf         chap8.pdf
chap5.pdf
```

From pdf to tiff

Without any bells and whistles, a simple bash script will suffice to extract high resolution `.tiff` files from `.pdf` inputs. The only caveat is the need to carefully format the page names so they can be joined together easily. This lead to:

```
# The first page number (output file name) for the current PDF
export INP_SCAN_NAME=toc_chap1_intro
output_index=1
pdftoppm -tiff "${INP_SCAN_NAME}.pdf" temp
i=0
for file in temp*.tif; do
    i=$((i+1))
    new_index=$(printf "%03d" $((output_index + i - 1)))
    new_file="page_${new_index}.tiff"
    mv "$file" "images/$new_file"
done
```

At this point we can plug the images directly into ScanTailor Advanced and work through the GUI.

OCR Setup

Working with the TIFF images which form the output at ScanTailor Advanced is a good bet. System packages needed are:

```
trizen -S tesseract tesseract-data-eng \
tesseract-data-is1 \
ghostscript unpaper pngquant \
img2pdf jbig2enc pdftk
```

Subsequently I ended up with a throwaway `conda` environment for using [OCRmyPDF](#) (like with all `python` tools).

```
micromamba create -p $(pwd)/.ocrtmp pip ocrmypdf
micromamba activate $(pwd)/.ocrtmp
```

Since `ocrmypdf` works on single PDF files or alternatively single images, one brute-force approach is simply:

```
cd out # where scantailor puts things
mkdir temp
for file in page*.tif; do
    base=$(basename "$file" .tif)
    ocrmypdf -l eng --output-type pdf "$file" "temp/${base}.pdf"
done
```

Which leads to a series of `.pdf` files.

Concatenation and Compression

[Ghostscript](#) is the best way to get a single reasonably **compressed** output from these files^[^fn:4]:

```
export OUTPUT_PDF_DPI=500 # good enough for me
# Or 600 for the Scantailor default
gs -q -dNOPAUSE -dBATCH -sDEVICE=pdfwrite \
  -dCompatibilityLevel=1.7 -dPDFSETTINGS=/ebook \
  -dEmbedAllFonts=true -dSubsetFonts=true \
  -dColorImageDownsampleType=/Bicubic \
  -dColorImageResolution=$OUTPUT_PDF_DPI \
  -dGrayImageDownsampleType=/Bicubic \
  -dGrayImageResolution=$OUTPUT_PDF_DPI \
  -dMonoImageDownsampleType=/Bicubic \
  -dMonoImageResolution=$OUTPUT_PDF_DPI \
  -sOutputFile="output_${OUTPUT_PDF_DPI}.pdf" \
  temp/*.pdf
```

Where the compatibility level is to [allow for linearization](#) [^fn:5] (needs [PDF 1.7](#)). Sometimes this leads to files which are still too large (Table ref:tbl:sizes).

Table 1: For a color book

DPI Size (in MB)

300 139.2

600 303.5

In such cases `pdfsizeopt` (from [here](#)) can be setup in the environment with:

```
mkdir $CONDA_PREFIX/tmp/pdfsizeopt
cd $CONDA_PREFIX/tmp/pdfsizeopt
wget -O pdfsizeopt_libexec_linux.tar.gz https://github.com/pts/pdfsizeopt/releases/download/2023-04-18/pdfsizeopt_libexec_linux-v9.tar.gz
tar xzvf pdfsizeopt_libexec_linux.tar.gz
rm -f pdfsizeopt_libexec_linux.tar.gz
wget -O pdfsizeopt.single https://raw.githubusercontent.com/pts/pdfsizeopt/master/pdfsizeopt.single
chmod +x pdfsizeopt.single
mv * $CONDA_PREFIX/bin && cd $CONDA_PREFIX/bin
ln -s pdfsizeopt.single $CONDA_PREFIX/bin/pdfsizeopt
rm -rf $CONDA_PREFIX/tmp/*
```

Which can then be used as:

```
pdfsizeopt output_600.pdf
```

The compression takes a while [^fn:6], but the results are completely lossless and the reduction was pretty neat, from 390 MB to 250 MB, though it took forever, and [Adobe Acrobat](#) reduced a 390 MB file to 108 MB in minutes.

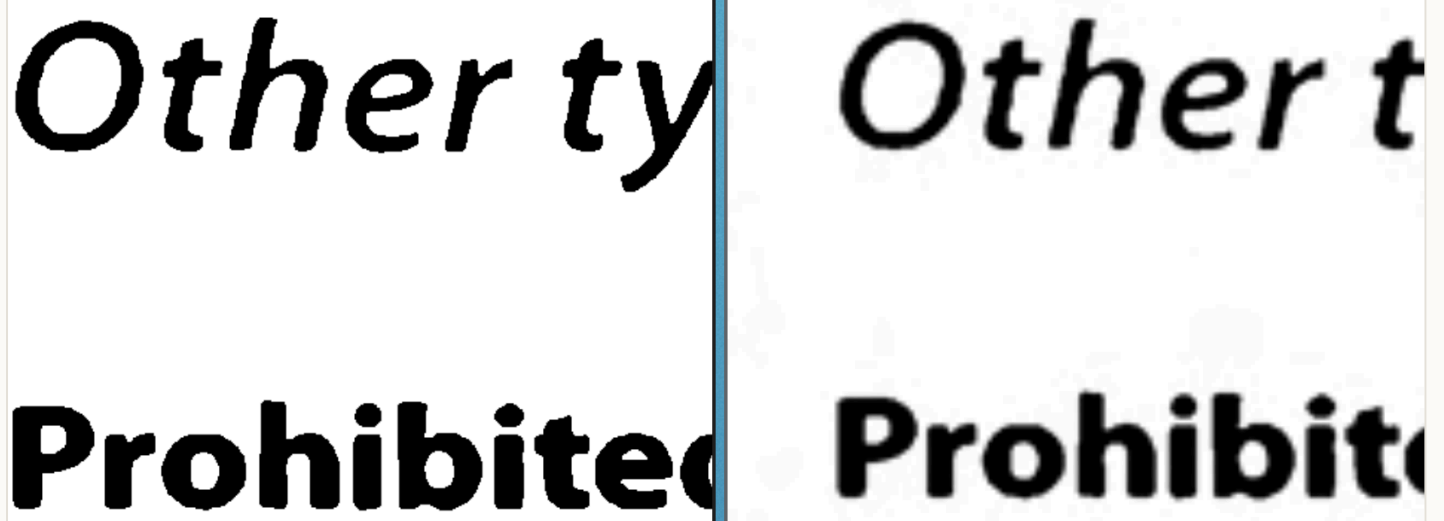


Figure 1: Lossless compression ('250 MB') on the left, lossy Acrobat compression ('108 MB') on the right

For handling the cover image I ended up with:

```
# from the ScanTailor 600 DPI image
# 39.3 MB
tiffcp -c zip:p9 cpage-1.tif cpage.tif # 19.7 MB
convert cpage.tif -compress JPEG -quality 90% cpage.jpg # 2.0 MB
```

Then I dumped it through a [web tool](#) [^fn:7](around 560 KB) before finishing up with `ocrmypdf` for the final pdf of 535 KB .

TOC Generation

Sometimes I feel too lazy and end up doing my table of contents editing in Master PDF editor [^fn:8]. That being said my most favored way for a fully fledged TOC with chapters and sub-headings is with the venerable [PDFtk](#):

```
cat toc.txt
BookmarkBegin
BookmarkTitle: Chapter 1
BookmarkLevel: 1
BookmarkPageNumber: 5

BookmarkBegin
BookmarkTitle: Section 1.1
BookmarkLevel: 2
BookmarkPageNumber: 6

BookmarkBegin
BookmarkTitle: Section 1.2
BookmarkLevel: 2
BookmarkPageNumber: 10
BookmarkBegin

BookmarkTitle: Chapter 2
BookmarkLevel: 1
BookmarkPageNumber: 15
...
```

This can be applied with:

```
pdftk input.pdf update_info toc.txt output output_with_toc.pdf
```

Sometimes I end up padding the `pdf` with a few extra pages to ensure the page numbers (printed) match the ones in the TOC.

Linearization

Since we targeted PDF 1.7, with linearization support [^fn:9]. We need [qpdf](#):

```
micromamba install qpdf
qpdf --linearize output_300.pdf olin300.pdf
```

This has seemingly no effect on size so it can be the last step.

Conclusions

This isn't my first time working with a scanned book. Felt like a lot more steps than before though. Some more bash magic could have automated extracting the images from scanner generated `pdf` files, and perhaps some other minor tweaks [^fn:10] or other tools [^fn:11], though for my purposes, this worked well enough. I was a little disappointed to not be able to hit optimal file sizes without web tools, but given the amount of time expended, it was probably to be expected.